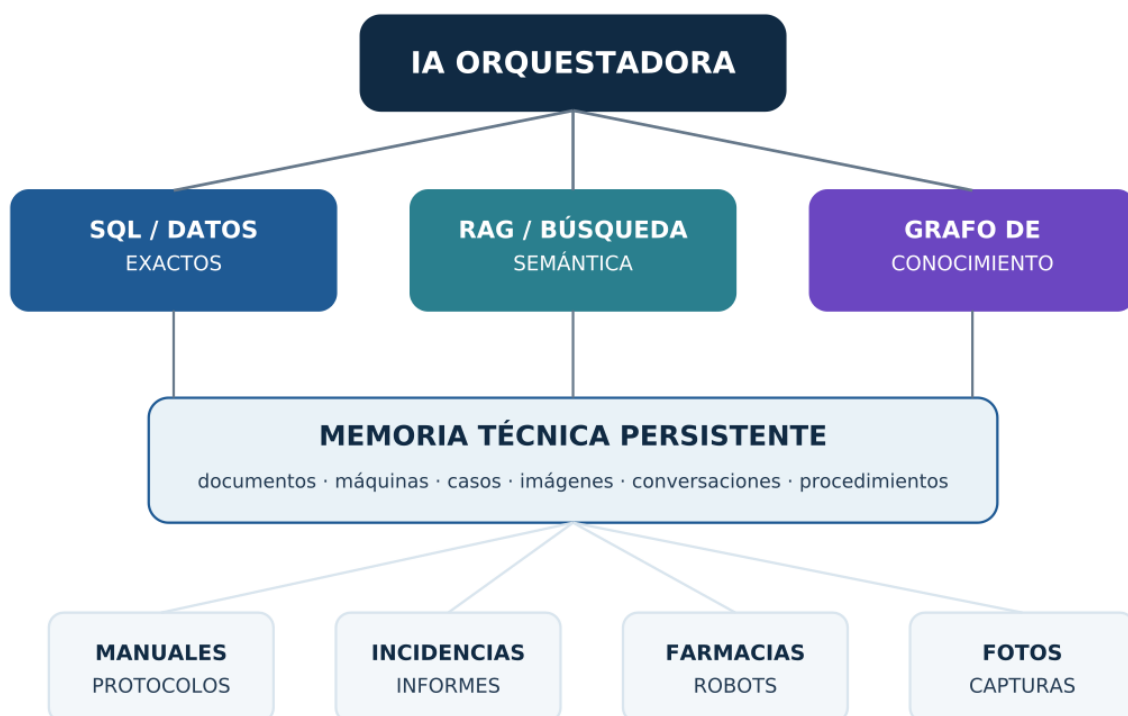


Sistema de IA con memoria técnica para asistencia sobre robots APObot

Arquitectura de conocimiento, RAG, memoria de casos, grafos, imágenes y razonamiento asistido para que una persona sin conocimiento técnico previo pueda saber qué mirar, por qué mirarlo y qué hacer a continuación.



PRINCIPIO DEL PROYECTO

La IA no debe limitarse a responder preguntas. Debe actuar como una capa de acceso al conocimiento técnico de la empresa: identificar el contexto de la farmacia y del robot, recuperar documentación y experiencia previa, relacionar evidencias y guiar al usuario hasta una comprobación o actuación concreta.

Documento técnico de arquitectura y propuesta de implementación · Septiembre 2026

1. Objetivo técnico

El objetivo es construir una aplicación de inteligencia artificial especializada en los robots APObot. Su característica principal será una memoria técnica persistente: el conocimiento no residirá únicamente en el modelo de lenguaje, sino en bases de datos, documentos, casos históricos, relaciones entre componentes y evidencias visuales que la IA consultará en cada interacción.

IDEA BASE

Un usuario puede no saber qué es un SGS, dónde está conectado, qué protocolo le corresponde o qué averías ha tenido antes. La aplicación debe transformar una frase humana en un proceso técnico guiado y trazable.

Ejemplo de entrada

«Me llaman de Farmacia X. Está fallando el SGS. Revisa los protocolos y dime qué está pasando.»

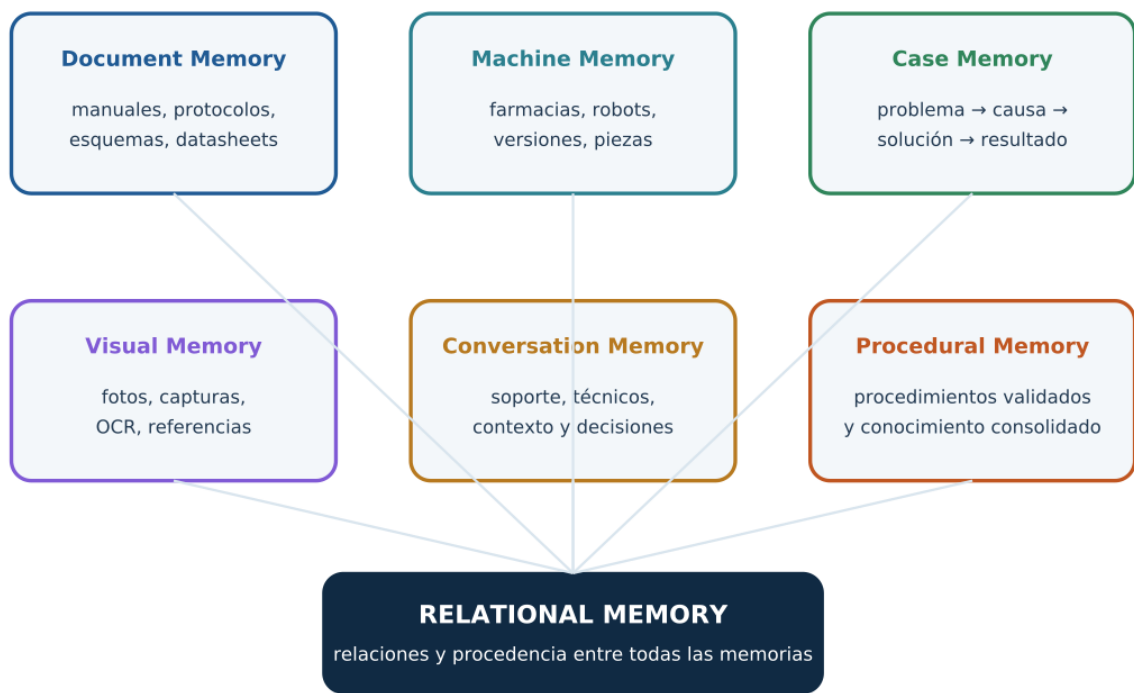
Para contestar bien, la IA necesita algo más que un PDF. Debe identificar la farmacia, saber qué robot tiene instalado, su versión y configuración, localizar qué es el SGS, consultar los protocolos aplicables, recuperar incidencias similares, revisar el historial de ese robot, encontrar informes técnicos validados y, si existen imágenes o capturas, incorporarlas como evidencia.



2. La IA no es la memoria

El modelo de lenguaje será la capa de interpretación y razonamiento. La memoria real estará fuera del modelo. Esto permite actualizar el conocimiento sin reentrenar la IA, conservar trazabilidad, aplicar permisos, versionar información y corregir conocimiento incorrecto.

Capa	Responsabilidad	Ejemplo
LLM / IA	Comprender la pregunta, razonar sobre evidencias y redactar la respuesta.	«Estas tres alarmas apuntan primero a comunicación PLC.»
PostgreSQL	Datos exactos y estructurados.	Qué robot tiene la farmacia, versión, piezas, incidencias.
Vector store / RAG	Encontrar información por significado.	«fallo de referencia» ≈ «no encuentra nullpunkt».
Case Memory	Recordar averías resueltas.	Problema → diagnóstico → causa → solución → resultado.
Knowledge Graph	Entender relaciones entre entidades.	Robot → SGS → PLC → entrada → protocolo → incidencia.
Object Storage	Conservar originales.	PDF, fotos, capturas, esquemas.
Visión/OCR	Extraer información de imágenes.	Leer una referencia o interpretar una captura.



3. Tipos de memoria que debe tener el sistema

Document Memory. Manuales de máquinas y componentes, protocolos, esquemas, documentación de PLC, tablas de parámetros, procedimientos, datasheets y despieces. Responde principalmente a «qué dice la documentación».

Machine Memory. Farmacias, robots, números de serie, versiones, configuración, componentes instalados, referencias, software y parámetros. Responde a «qué tiene exactamente esta máquina».

Case Memory. Incidencias e informes técnicos convertidos a una estructura reutilizable: problema, síntomas, diagnóstico, causa, solución y resultado. Responde a «qué nos ocurrió antes y qué funcionó».

Conversation Memory. Conversaciones con soporte y técnicos. Conserva el contexto, pero no todo lo conversado se convierte automáticamente en verdad técnica.

Visual Memory. Fotografías, capturas, OCR, referencias detectadas y asociaciones con componentes, incidencias y documentos.

Procedural Memory. Procedimientos consolidados y validados a partir de documentación oficial y experiencia repetida.

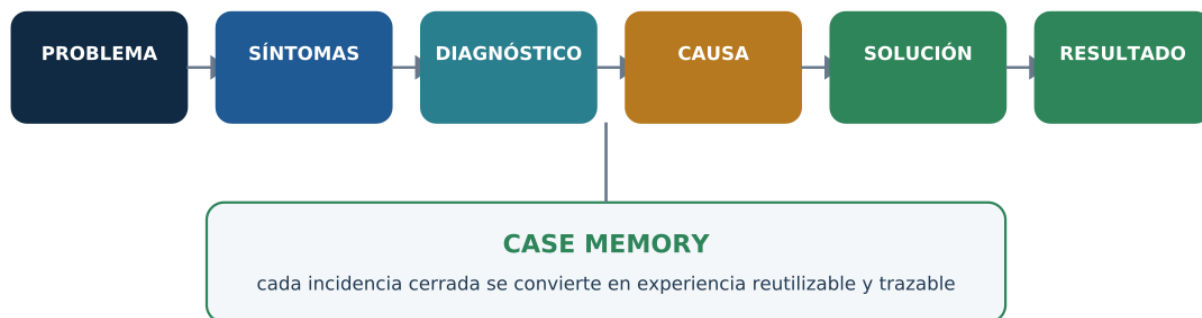
Relational Memory. Las conexiones entre todas las anteriores: una farmacia tiene un robot; ese robot lleva un SGS; el SGS usa un PLC; una incidencia afectó a ese componente; una solución funcionó; una foto pertenece al informe.

REGLA DE CALIDAD

La memoria debe guardar siempre procedencia, versión, fecha, nivel de confianza y estado de validación. Una hipótesis de soporte no puede tener la misma autoridad que un manual vigente o un informe técnico cerrado.

4. Case Memory: convertir averías en experiencia reutilizable

La información más valiosa a largo plazo será la experiencia acumulada. Cada incidencia cerrada debe poder convertirse en un caso técnico estructurado.



Campo	Ejemplo
Problema	SGS no comunica con el sistema.
Síntomas	Alarmas intermitentes; reinicio sin efecto.
Contexto	Farmacia X · APObot modelo Y · versión Z.
Diagnóstico	Pérdida de comunicación entre SGS y PLC.
Causa	Conector X con falso contacto.
Solución	Revisión, reconexión y prueba de ciclos.
Resultado	Correcto; incidencia no reproducida.
Evidencias	Informe IT-204; fotos 1-3; protocolo SGS v4.

Cuando aparezca un problema nuevo, el sistema podrá buscar casos semánticamente parecidos y además filtrar por modelo, versión, componente, error o farmacia. La solución no se considera validada por el mero hecho de que la IA la proponga; debe conservarse el resultado real de la intervención.

5. RAG: para qué sirve y cuándo utilizarlo

RAG (Retrieval-Augmented Generation) consiste en recuperar información relevante antes de pedir al modelo que responda. El modelo recibe la pregunta junto con fragmentos de manuales, protocolos, informes o casos relacionados. De esta forma la respuesta se apoya en información interna y actualizable.

Preguntas donde RAG aporta mucho valor

Pregunta	Qué recuperaría RAG
«¿Qué comprobaciones indica el protocolo del SGS?»	Secciones del protocolo SGS relacionadas con diagnóstico y comprobaciones.
«¿Cómo se parametriza este motor?»	Manual del motor y sección de parametrización correspondiente.
«¿Qué dice el manual sobre el error 342?»	Fragmentos que contienen el error y procedimientos relacionados.
«¿Hemos tenido averías parecidas?»	Informes e incidencias semánticamente similares.
«¿Qué procedimiento existe para cambiar esta pieza?»	Procedimiento interno y manual de sustitución.

El RAG debe ser híbrido: combinar búsqueda semántica mediante embeddings con búsqueda léxica/full-text y filtros por metadatos. Esto es especialmente importante para referencias, códigos de error, nombres exactos y números de pieza.

LIMITACIÓN DEL RAG

RAG encuentra texto relevante, pero no entiende por sí solo todas las relaciones del dominio. Puede localizar un manual del SGS, pero para responder «qué SGS lleva exactamente esta farmacia y qué incidencias tuvo ese mismo componente» necesita datos estructurados y/o un grafo.

6. Opciones tecnológicas para RAG y búsqueda vectorial

La recomendación inicial es empezar con PostgreSQL + pgvector. Permite mantener datos operativos y embeddings en el mismo sistema, reducir infraestructura y aplicar filtros SQL sobre modelo, versión, componente o farmacia. pgvector soporta búsqueda exacta y aproximada con índices HNSW e IVFFlat.

Opción	Ventajas	Cuándo elegirla
PostgreSQL + pgvector	Una sola base; SQL + metadatos + vectores; HNSW/IVFFlat; fácil de integrar con EF Core.	Opción recomendada para MVP y primera producción.
Qdrant	Motor vectorial especializado; filtros de payload; búsqueda híbrida dense+sparse y consultas multietapa.	Si el volumen vectorial o la complejidad de retrieval crecen mucho.
Semantic Kernel Vector Store	Abstracción .NET sobre distintos almacenes; integración natural con agentes/RAG.	Como capa de software para evitar acoplarse a un proveedor concreto; varias APIs siguen evolucionando.
PostgreSQL Full Text + pgvector	Combina términos exactos y semántica dentro de PostgreSQL.	Muy útil para códigos de error, referencias y lenguaje técnico.

PostgreSQL ofrece full-text search mediante tsvector/tsquery y recomienda índices GIN para búsquedas textuales frecuentes. Qdrant, como alternativa especializada, soporta búsqueda híbrida semántica y léxica y filtros sobre payload. Estas capacidades permiten construir un retrieval que no dependa únicamente de similitud vectorial.

Decisión propuesta

FASE INICIAL

PostgreSQL + pgvector + Full Text Search. Mantener Qdrant como alternativa de escalado. En .NET, encapsular el acceso mediante interfaces propias o conectores de Semantic Kernel para poder sustituir el backend de búsqueda sin rehacer la lógica de negocio.

7. Knowledge Graph: por qué hace falta

El dominio de un robot de farmacia es naturalmente relacional. Un documento aislado puede decir cómo funciona un componente; el grafo permite saber cómo ese componente se relaciona con una máquina concreta, una versión, un PLC, una incidencia, una pieza y una solución.

Ejemplo conceptual

Farmacia X → HAS_ROBOT → AP0bot #184
AP0bot #184 → HAS_COMPONENT → SGS #7
SGS #7 → CONNECTED_TO → PLC Beckhoff
SGS #7 → DOCUMENTED_BY → Protocolo SGS v4
SGS #7 → AFFECTED_BY → INC-204
INC-204 → SOLVED_BY → Reconexión conector X

Con esta estructura podemos hacer consultas multi-salto: «¿Qué incidencias han afectado a componentes conectados al mismo PLC que el SGS de esta farmacia?» o «¿Qué piezas instaladas en esta generación tienen procedimientos de sustitución diferentes?»



8. Opciones tecnológicas para el grafo de conocimiento

Opción	Fortalezas	Encaje
Neo4j	Maduro, Cypher, ecosistema amplio, vector indexes y paquete oficial GraphRAG.	Recomendación principal si el grafo pasa a ser una pieza central.
Memgraph	Cypher, arquitectura de alto rendimiento, modos in-memory/on-disk, herramientas de visualización y algoritmos.	Alternativa muy interesante si se prioriza tiempo real y despliegue controlado.
PostgreSQL (Entities + Relations)	Sin nueva infraestructura; relaciones simples modeladas con tablas.	Recomendado para comenzar antes de justificar una base de grafos dedicada.
ArangoDB / multimodelo	Documento + grafo en un mismo motor.	Alternativa si se desea consolidar modelos de datos, pero aumenta el cambio tecnológico.

La propuesta es no introducir Neo4j el primer día. En la fase inicial se pueden almacenar entidades y relaciones en PostgreSQL. Cuando las consultas multi-hop, el volumen de relaciones o GraphRAG aporten valor demostrable, replicar o migrar esa capa a Neo4j o Memgraph.

Neo4j dispone de herramientas oficiales GraphRAG con retrievers vectoriales y consultas sobre el grafo. Memgraph ofrece Cypher y herramientas orientadas a contexto conectado y razonamiento multi-hop. La elección final debe validarse con un prototipo sobre datos reales.

CRITERIO DE DECISIÓN

Si la mayoría de preguntas se resuelven con documentos y filtros, PostgreSQL puede bastar. Si empezamos a preguntar continuamente por cadenas de relaciones entre robot, componente, versión, error, incidencia, pieza y solución, el grafo deja de ser opcional y pasa a ser una herramienta de primera clase.

9. Cuándo RAG no basta: ejemplos que requieren combinar tecnologías

Pregunta	RAG	SQL	Grafo	Imagen
«¿Qué dice el protocolo SGS sobre pérdida de comunicación?»	Sí	No	No	No
«¿Qué SGS tiene instalado Farmacia X?»	No	Sí	Opcional	No
«¿Qué protocolo corresponde al SGS exacto de Farmacia X?»	Sí	Sí	Sí	No
«¿Ha ocurrido antes en ese mismo robot?»	Sí	Sí	Opcional	No
«¿Qué componentes conectados al mismo PLC han fallado antes?»	Parcial	Parcial	Sí	No
«Identifica esta referencia de una foto y busca sus averías anteriores.»	Sí	Sí	Sí	Sí
«Tengo estas tres alarmas: ¿qué causa común las conecta?»	Sí	Sí	Sí	Opcional

La arquitectura no busca usar todas las tecnologías en todas las preguntas. El orquestador debe decidir qué herramientas necesita. Una consulta documental simple puede resolverse solo con RAG. Una consulta sobre una farmacia concreta necesita SQL. Una pregunta relacional compleja puede activar el grafo. Una fotografía activa visión/OCR.

10. Flujo específico para soporte

Soporte necesita convertir una llamada poco precisa en una secuencia de comprobaciones útiles. La IA debe reducir la dependencia del conocimiento individual y guiar incluso a una persona que no conozca profundamente la máquina.

Ejemplo

«Me llaman de Farmacia Central. Dicen que el SGS falla y el robot no termina el ciclo.»

Paso 1 - contexto. La aplicación identifica la farmacia y recupera el robot instalado, versión, software y SGS asociado.

Paso 2 - conocimiento oficial. Recupera el protocolo SGS y las secciones de diagnóstico aplicables a esa configuración.

Paso 3 - experiencia. Busca incidencias de esa farmacia, de ese robot y casos similares en el resto de instalaciones.

Paso 4 - relaciones. Comprueba componentes conectados, PLC, errores asociados y causas históricas.

Paso 5 - interacción. Si falta información, no inventa: pregunta. «¿Aparece el LED X?», «¿Qué código muestra la pantalla?», «Adjunta una foto del módulo».

Paso 6 - salida. Presenta qué mirar físicamente, en qué orden, qué resultado esperar y cuándo escalar a técnico.

SALIDA ESPERADA

«Para este modelo, el protocolo indica comprobar primero alimentación y comunicación del SGS. En 12 casos similares, 7 fueron conector/comunicación, 3 alimentación y 2 parametrización. En esta misma farmacia hubo un caso equivalente en marzo. Comprueba A → B → C. Si el LED X permanece apagado, adjunta una fotografía del módulo antes de continuar.»

11. Flujo específico para técnicos

El técnico necesita mayor profundidad y acceso a documentación, histórico, referencias, parámetros e imágenes. La misma memoria puede ofrecer una interfaz más técnica y menos guiada.

Ejemplos de preguntas

Pregunta del técnico	Qué debería hacer el sistema
«He cambiado el motor del eje S. ¿Qué parámetros tengo que tocar?»	Identificar modelo/versión, recuperar manual del motor, tabla de parámetros y casos anteriores de sustitución.
«¿Dónde está conectado el sensor de referencia?»	Resolver componente → esquema → terminal → PLC; mostrar página/plano.
«¿Qué referencia lleva esta correa?»	Consultar configuración/despiece del robot concreto y devolver referencia con fuente.
«Estas alarmas aparecen juntas. ¿Las hemos visto antes?»	Buscar coocurrencias en casos, recuperar causas y resultados.
«Enséñame fotos de intervenciones anteriores en este SGS.»	Recuperar Visual Memory filtrada por componente/modelo/caso.

El técnico también alimenta el sistema. Al cerrar un informe, el pipeline de conocimiento extrae problema, síntomas, diagnóstico, causa, solución, piezas, parámetros, fotos y resultado. El técnico valida el informe; después el caso pasa a formar parte de la memoria.

12. Imágenes, capturas y OCR

Las fotografías no deben quedar como adjuntos opacos. Deben convertirse en objetos consultables y relacionados con el resto de la memoria.

Proceso	Resultado
Guardar original	Trazabilidad y evidencia sin pérdida.
OCR	Texto de etiquetas, referencias, pantallas y códigos.
Descripción multimodal	Descripción técnica de lo visible.
Etiquetado	Componente, zona, robot, incidencia.
Embedding visual/textual	Búsqueda de imágenes similares.
Entity linking	Relacionar una referencia detectada con la pieza existente.

Ejemplo: el técnico fotografía una placa donde aparece una referencia. OCR extrae la referencia; SQL localiza la pieza; el grafo determina en qué modelos se instala; RAG recupera el manual; Case Memory encuentra averías de esa misma referencia. Una sola imagen puede activar varias capas de conocimiento.

Para OCR/visión se puede comenzar con servicios cloud o librerías especializadas. Si se necesita procesamiento avanzado o modelos locales, se recomienda un microservicio Python aislado del núcleo .NET.

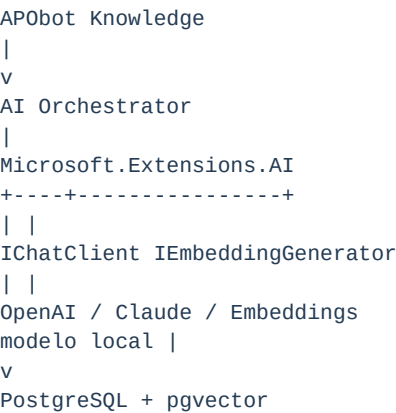
12.1. Microsoft.Extensions.AI y elección del modelo de IA

Microsoft.Extensions.AI no es un modelo de inteligencia artificial. Es la capa de abstracción de .NET que utilizaremos entre nuestra aplicación y el proveedor real de IA. Su función es evitar que el sistema de memoria, RAG, herramientas y lógica de negocio quede acoplado a OpenAI, Anthropic/Claude o a un modelo local.

POR QUÉ ES IMPORTANTE

La memoria del proyecto debe pertenecer a nuestra aplicación, no al proveedor del modelo. Si mañana cambiamos de OpenAI a Claude, o decidimos ejecutar un modelo local, no deberíamos reconstruir PostgreSQL, RAG, Case Memory, el grafo ni los flujos de soporte.

Arquitectura propuesta



Las dos abstracciones principales que nos interesan son IChatClient, para enviar conversaciones y contexto al modelo de razonamiento, e IEmbeddingGenerator, para generar los vectores que utiliza la búsqueda semántica. La aplicación trabajará contra estas interfaces, no contra llamadas repartidas por todo el código a una API concreta.

Pieza	Responsabilidad en el proyecto
IChatClient	Modelo que interpreta el problema, utiliza herramientas, contrasta evidencias y explica al humano qué mirar.
IEmbeddingGenerator	Convierte manuales, casos y consultas en vectores para búsqueda semántica.
Middleware / DI	Permite telemetría, logging, caché, resiliencia y sustitución de implementaciones.
APObot.AI	Nuestra capa propia: prompts, herramientas, RAG, políticas, fuentes y reglas del dominio.

El modelo no conocerá por sí mismo la empresa. Antes de responder, nuestro sistema le entregará el contexto que necesite: farmacia, robot, versión, manuales recuperados por RAG, casos históricos, relaciones del grafo, imágenes y procedimientos.

12.2. ¿OpenAI, Claude o un modelo local?

La arquitectura debe admitir los tres escenarios. No conviene convertir la elección del primer modelo en una dependencia permanente del producto.

Opción	Ventajas	Consideraciones	Uso propuesto
OpenAI API	Modelos potentes, tool calling, multimodalidad y ecosistema maduro.	Coste por uso y servicio externo.	Candidato inicial para razonamiento y pruebas.
Anthropic Claude API	Muy buen trabajo con contexto largo, documentación, razonamiento y tool use.	Coste por uso y servicio externo.	Candidato inicial; comparar con OpenAI.
Modelo local (Ollama / servidor propio)	Control del entorno y posibilidad de inferencia on-premise.	Hardware, operación, actualizaciones y calidad dependiente del modelo.	Alternativa estratégica para privacidad o tareas concretas.
Arquitectura híbrida	Modelos diferentes según tarea, coste o sensibilidad.	Más complejidad de routing y evaluación.	Evolución si aporta ventajas medibles.

No es obligatorio utilizar el mismo proveedor para todo. Por ejemplo, podemos utilizar un modelo potente de OpenAI o Claude mediante IChatClient para razonamiento y un modelo de embeddings distinto mediante IEmbeddingGenerator. Incluso podríamos ejecutar embeddings localmente y mantener el razonamiento principal en cloud.

Ejemplo real del flujo

Pregunta: «Estoy en Farmacia X. El SGS da tres errores. La alimentación está correcta. Revisa protocolos, averías anteriores de este robot y otros casos similares y dime qué comprobar después.»

1. SQL identifica máquina y configuración. 2. RAG recupera los protocolos adecuados. 3. Case Memory busca averías similares. 4. El grafo relaciona SGS, PLC, alarmas y componentes. 5. Si hay fotografía, visión/OCR aporta evidencia. 6. Solo entonces el contexto se entrega al modelo mediante IChatClient. 7. El modelo razona y devuelve una explicación guiada con fuentes.

PROPUESTA DE ARRANQUE

No elegir por intuición entre Claude y OpenAI. Crear un benchmark con aproximadamente 50-100 casos técnicos reales: preguntas de manuales, diagnóstico, uso de herramientas, imágenes y relaciones. Medir precisión, citas correctas, capacidad de pedir información faltante, latencia y coste. Elegir el modelo principal a partir de esos resultados.

La recomendación es comenzar con una API cloud de alta capacidad para conseguir la mejor calidad posible durante el MVP, manteniendo desde el primer día la interfaz desacoplada. Un modelo local se evaluará en paralelo para tareas donde privacidad, coste o funcionamiento on-premise sean especialmente importantes.

En resumen: Microsoft.Extensions.AI es el conector; OpenAI, Claude u Ollama son posibles motores; nuestra memoria, RAG, grafo y reglas son el producto.

13. Stack tecnológico propuesto

Área	Tecnología propuesta	Motivo
Frontend	Blazor + MudBlazor	Interfaz en C# y componentes UI consistentes.
Backend	ASP.NET Core / .NET 10	API, seguridad, lógica de negocio, workers y servicios.
ORM	Entity Framework Core	Modelo relacional, migraciones y acceso a PostgreSQL.
IA	Microsoft.Extensions.AI	Abstracciones IChatClient/IEmbeddingGenerator y desacoplamiento del proveedor.
Orquestación avanzada	Semantic Kernel (selectivo)	Plugins, agentes y conectores de búsqueda; usar solo donde aporte valor.
Datos	PostgreSQL	Fuente estructurada principal.
Vector search	pgvector	Embeddings en PostgreSQL; HNSW/IVFFlat.
Text search	PostgreSQL FTS	Códigos, referencias y términos exactos.
Archivos	S3-compatible object storage	PDF, fotos, esquemas y originales.
Grafo	PostgreSQL inicialmente; Neo4j/Memgraph después	Evitar complejidad prematura; escalar a multi-hop/GraphRAG.
Visión/OCR	Servicio auxiliar Python o proveedor externo	Ecosistema CV/OCR sin convertir Python en el núcleo.
Jobs	BackgroundService + cola si crece	Ingesta, OCR, embeddings, consolidación y reindexado.

Microsoft.Extensions.AI ofrece actualmente abstracciones genéricas como `IChatClient` e `IEmbeddingGenerator`, adecuadas para desacoplar el dominio del proveedor de IA. Semantic Kernel dispone de componentes de búsqueda/RAG y conectores vectoriales, aunque algunas APIs relacionadas con agentes y vector stores siguen marcadas como preliminares/experimentales; por ello conviene encapsularlas detrás de interfaces propias.

14. Dependencias y módulos de software

La solución debe dividirse por responsabilidades para que la memoria no quede mezclada con la interfaz o con un proveedor de IA concreto.

```
APObot.Knowledge.sln

src/
APObot.Web # Blazor / UI
APObot.Api # ASP.NET Core API
APObot.Domain # Robot, Pharmacy, Component, Incident, Case...
APObot.Application # casos de uso y workflows
APObot.Infrastructure # EF Core, PostgreSQL, storage
APObot.AI # RAG, retrieval, tools, reasoning
APObot.Knowledge # entities, relations, consolidation
APObot.Workers # ingestion, OCR, embeddings, indexing

services/
apobot-vision/ # opcional: Python / OCR / CV
```

Dependencias .NET recomendadas

Dependencia / familia	Uso
Microsoft.EntityFrameworkCore + proveedor PostgreSQL	Persistencia relacional y migraciones.
Microsoft.Extensions.AI	Chat y embeddings mediante interfaces comunes.
Semantic Kernel (opcional)	Plugins, agentes, retrieval y experimentación con RAG.
Cliente pgvector/Npgsql	Tipos vectoriales y consultas de similitud.
Cliente S3 compatible	Documentos e imágenes.
OpenTelemetry	Trazas, latencia, coste y observabilidad del pipeline de IA.
Polly / resilience handlers	Reintentos, timeouts y circuit breakers en servicios externos.

15. Pipeline de ingestión de conocimiento

Un PDF o informe no debe entrar directamente al prompt. Debe pasar por un pipeline reproducible.

Etapas	Qué ocurre
1. Registro	Se guarda archivo, hash, tipo, versión, propietario y procedencia.
2. Extracción	Texto nativo; OCR solo si es necesario; tablas e imágenes por separado.
3. Estructuración	Capítulos, páginas, secciones, tablas, parámetros y referencias.
4. Chunking semántico	Fragmentos por estructura lógica, no cortes arbitrarios.
5. Metadatos	Modelo, versión, componente, fabricante, vigencia, permisos.
6. Embeddings	Representación vectorial para búsqueda semántica.
7. Full-text	Índice léxico para códigos y términos exactos.
8. Extracción de entidades	Robot, componente, pieza, error, parámetro, procedimiento.
9. Relaciones	Crear o proponer enlaces entre entidades.
10. Validación	Confianza, contradicciones y revisión cuando corresponda.

El contenido original nunca se sustituye por la interpretación de la IA. El original permanece como fuente primaria y cada conocimiento derivado conserva una referencia a él.

16. Autoridad, confianza y prevención de conocimiento incorrecto

La memoria debe distinguir hechos, hipótesis, inferencias y procedimientos validados. Sin esta separación, una conversación informal puede contaminar el sistema.

Tipo de fuente	Tratamiento
Manual vigente / documentación oficial	Alta autoridad; aplicar versión y contexto.
Procedimiento interno aprobado	Alta autoridad; control de versiones.
Informe técnico cerrado	Evidencia de campo validada.
Incidencia abierta	Describe síntomas, no necesariamente causa.
Conversación de soporte	Contexto útil; afirmaciones no verificadas.
Sugerencia de IA	Hipótesis; nunca se promueve sola a hecho.

Ejemplo: soporte escribe «creo que es el motor». La memoria guarda una hipótesis. El técnico comprueba que el motor está correcto y encuentra un sensor defectuoso. El caso cerrado registra el sensor como causa diagnosticada y la hipótesis inicial como descartada.

También debe existir detección de contradicciones: si un manual nuevo cambia un parámetro o un informe usa un valor diferente al oficial, el sistema no debe fusionarlos silenciosamente.

17. Orquestador de IA y herramientas

El orquestador decide qué memoria consultar. No todas las preguntas deben enviar toda la base de conocimiento al modelo.

Herramientas internas posibles

```
GetPharmacyContext(pharmacyId)
GetRobotConfiguration(robotId)
SearchManuals(query, filters)
SearchCases(query, filters)
GetRobotHistory(robotId)
GetComponentRelations(componentId)
GetPartReference(componentId)
SearchTechnicalImages(query, filters)
GetProcedure(component, action, version)
RegisterSuggestedSolution(caseId, solution)
RegisterOutcome(caseId, result)
```

Ejemplo de decisión: «¿Qué referencia tiene esta correa?» probablemente requiere SQL y quizá despiece documental; no necesita un recorrido GraphRAG completo. «¿Qué causas comunes conectan estas tres alarmas en este modelo?» puede requerir RAG + casos + grafo.

OBJETIVO DE DISEÑO

La IA debe utilizar herramientas, no inventar datos. Si no encuentra la referencia o no puede determinar la versión del robot, debe decirlo y pedir la información necesaria.

18. Cómo debe responder la IA a una persona no experta

El formato de respuesta es parte del producto. La IA debe traducir conocimiento técnico a acciones verificables sin ocultar la evidencia.

Bloque de respuesta	Ejemplo
Contexto identificado	Farmacia X · APObot #184 · versión Y · SGS ref. Z.
Qué puede estar ocurriendo	Hipótesis ordenadas por evidencia, no por intuición.
Qué mirar ahora	«Localiza el módulo SGS en... Comprueba el LED X.»
Resultado esperado	«Si está verde fijo, alimentación correcta; si está apagado...»
Siguiente paso	Qué hacer según el resultado observado.
Evidencia	Manual, protocolo, caso histórico, fotografía.
Confianza	Alta/media/baja y motivo.
Escalado	Cuándo dejar de guiar y enviar técnico.

Esto materializa la idea principal: una persona sin conocimiento previo debe poder seguir la respuesta y saber exactamente qué está mirando. La IA actúa como interfaz entre el conocimiento técnico acumulado y el usuario.

19. Roadmap recomendado

Fase	Objetivo	Resultado demostrable
1. Modelo de dominio	PostgreSQL, farmacias, robots, componentes, documentos, incidencias.	Base coherente y versionada.
2. Document Memory + RAG	Ingesta de manuales/protocolos, embeddings, full-text y citas.	Preguntas fiables sobre documentación.
3. Case Memory	Normalizar informes e incidencias resueltas.	Buscar problemas similares y soluciones.
4. Asistente soporte	Contexto de farmacia + RAG + casos + guía paso a paso.	Usuario no experto sabe qué comprobar.
5. Asistente técnico	Histórico, parámetros, referencias, procedimientos.	Asistencia avanzada en intervención.
6. Visual Memory	Fotos, OCR y referencias.	Preguntas con evidencia visual.
7. Knowledge Graph	Entidades y relaciones multi-hop.	Consultas relacionales complejas.
8. GraphRAG	Combinar retrieval textual, vectorial y grafo.	Razonamiento conectado con trazabilidad.
9. Consolidación	Proponer procedimientos a partir de patrones.	Memoria institucional creciente.
10. Analítica predictiva	Patrones de fallo y anomalías.	Base para mantenimiento predictivo.

POR DÓNDE EMPEZAR

Primero construir bien el modelo de datos y la ingestión documental. Después demostrar un RAG híbrido con citas sobre manuales reales. El siguiente salto de valor es Case Memory sobre incidencias cerradas. El grafo debe incorporarse cuando las relaciones reales del dominio ya estén suficientemente modeladas.

20. Ejemplo completo: fallo de SGS en una farmacia

Entrada: «Me llaman de Farmacia X. Está fallando el SGS. Revisa los protocolos y dime qué está pasando.»

1. Resolución de contexto. SQL identifica Farmacia X, el robot instalado, versión, número de serie y configuración. Se determina qué SGS corresponde.
2. Documentación. RAG recupera el protocolo SGS correcto y las secciones de diagnóstico, alimentación, comunicación y alarmas.
3. Histórico. Case Memory recupera incidencias de ese robot y casos similares de otros robots, priorizando misma versión y mismo componente.
4. Relaciones. El grafo conecta SGS → PLC → alimentación → errores → piezas → procedimientos y permite buscar causas compartidas.
5. Evidencia visual. Si soporte adjunta una captura o foto, OCR/visión identifica códigos, LEDs o referencias y los enlaza con la memoria.
6. Razonamiento. El sistema contrasta protocolo oficial con experiencia histórica. Si existe conflicto, lo muestra; no lo oculta.
7. Respuesta guiada. La IA explica dónde está el elemento, qué comprobar, qué debería observar el usuario y qué hacer según cada resultado.

EJEMPLO DE RESPUESTA

Contexto: he identificado el SGS correspondiente al APObot de Farmacia X. Primera comprobación: localiza el módulo SGS y revisa el indicador de alimentación X. El protocolo de esta versión indica que debe permanecer [...]. En los casos históricos más parecidos, la causa más frecuente fue [...]. Si el indicador está [...], continúa con [...]. Si no puedes confirmar el estado, adjunta una fotografía del módulo. Fuentes: Protocolo SGS v4, sección 3.2; INC-204; IT-204. Confianza: media hasta confirmar el estado del indicador.

Este flujo resume la filosofía completa: la IA no sustituye el conocimiento técnico; lo organiza, lo recupera, lo conecta y lo convierte en instrucciones comprensibles.

21. Arquitectura recomendada y decisión técnica

La arquitectura recomendada para comenzar es .NET 10 + Blazor + ASP.NET Core + PostgreSQL + pgvector + full-text search + object storage + Microsoft.Extensions.AI. Sobre esta base se construyen Document Memory, Machine Memory y Case Memory. El grafo se modela inicialmente en PostgreSQL y se evalúa Neo4j o Memgraph cuando las consultas relacionales lo justifiquen. Python queda como servicio auxiliar para OCR, visión o ML especializado.

La clave no es escoger una única tecnología «de IA». La calidad provendrá de asignar a cada herramienta el problema que resuelve mejor: SQL para hechos exactos; RAG para conocimiento documental; Case Memory para experiencia; grafo para relaciones; visión para evidencias visuales; LLM para comprender, razonar y explicar.

CRITERIO DE ÉXITO

El sistema será útil cuando una persona que no conoce la máquina pueda formular el problema en lenguaje natural y la aplicación sea capaz de llevarla desde «no sé qué mirar» hasta una comprobación concreta, sustentada por conocimiento técnico verificable.

Referencias técnicas consultadas

- Microsoft Learn - Microsoft.Extensions.AI: IChatClient, IEmbeddingGenerator y contenido multimodal.
- Microsoft Learn - Semantic Kernel: text search, RAG y conectores de vector store.
- pgvector - búsqueda vectorial para PostgreSQL, índices HNSW e IVFFlat.
- PostgreSQL - Full Text Search, tsvector/tsquery e índices GIN.
- Qdrant - búsqueda híbrida, filtros y consultas multi-stage.
- Neo4j - GraphRAG, vector retrieval y Cypher.
- Memgraph - base de grafos Cypher y herramientas para contexto conectado.

Nota: algunas APIs de Semantic Kernel relacionadas con agentes/RAG/vector stores continúan evolucionando. La arquitectura propuesta las encapsula para evitar acoplamiento innecesario.